



Nombre:

Grupo:

Pregunta 1 (2 puntos):

Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

- [] En un árbol B, cualquier borrado implica acceder siempre a un nodo hoja.
- [] En la dispersión abierta, el agrupamiento secundario no se produce, pero el primario sí.
- [] La operación de liberar recursos al eliminar un árbol AVL es $O(\log n)$.
- [] En tablas de dispersión con cubetas, si al buscar llegamos a un hueco no siempre se detiene la búsqueda.
- [] La búsqueda de un punto es siempre más eficiente en una malla regular que en un quadtree.
- [] Un grafo poco denso(con pocas aristas entre vértices) es

- más eficiente en espacio el implementarlo utilizando listas de adyacencia que matrices.
- [] Un heap puede no estar balanceado si los valores se insertan en orden contrario al que se desea recuperar.
- [] Tanto las colas como las colas con prioridad se implementan igual de eficientes con vectores que con listas enlazadas. La única diferencia viene de si conocemos el tamaño máximo o no.
- [] El deque permite inserciones eficientes en cualquier posición.
- [] En las listas doblemente enlazadas, la inserción en la posición anterior a la apuntada por un iterador es $O(1)$.

Pregunta 2 (2 puntos):

Insertar (I) o borrar (B) la siguiente secuencia numérica en una tabla hash cerrada de tamaño 11 con resolución de colisiones cuadrática $h(x) = (x + i^2) \% 11$. Se pueden unificar varios pasos en uno pero quedando claro en todo momento los estados: vacío, ocupado y disponible. Secuencia: I(3), I(19), I(26), I(21), I(36), I(10), B(3), I(8), B(19), I(14), I(18). Indicad si se ha producido algún tipo de agrupamiento, de qué tipo y en qué casos..

Pregunta 3 (3 puntos):

- (a) Implemente la primitiva *T Heap<T>::menorPrioridad()* para los Heap la cual debe devolver el elemento de menor prioridad que esté almacenado en la estructura.
- (b) Implemente la primitiva para listasEnlazadas *void*

Split(ListaEnlazada<T> impares, ListaEnlazada<T> pares).
Esta primitiva recibe dos listas: impares y pares. En la primera almacenará los elementos situados en posiciones impares de la lista y en la segunda (pares) los valores en posiciones pares.

Pregunta 4 (3 puntos):

En una remota región de la amazonia, los investigadores han determinado que existen aún numerosas subespecies de insectos no catalogadas. Paradójicamente, los nativos de la zona que son grandes conocedores del territorio, se han incorporado rápidamente a las nuevas tecnologías y los teléfonos inteligentes, gracias a la conexión a Internet de sus poblados. Por esta razón se quiere diseñar una aplicación móvil colaborativa llamada **AnimaLoca** para que los habitantes de la zona se animen a localizar especies de animales, en concreto insectos, y ayudar así al proceso de catalogación. No sólo se pretende conocer nuevas especies, sino determinar la posición geográfica donde éstas se encuentran para entender mejor su hábitat, por lo que se pueden repetir los registros para el mismo animal. El diseño de clases destacado para esta app es el siguiente:

Insecto: corresponde a una subespecie dentro de cada una de las especies de insectos. Se supone que los participantes conocen las grandes especies de insectos: mosquitos, polillas, cucarachas, etc. Lo que no es tan fácil es conocer la subespecie de cada uno de ellos. De cada insecto se guarda una fotografía considerada oficial dentro del sistema.

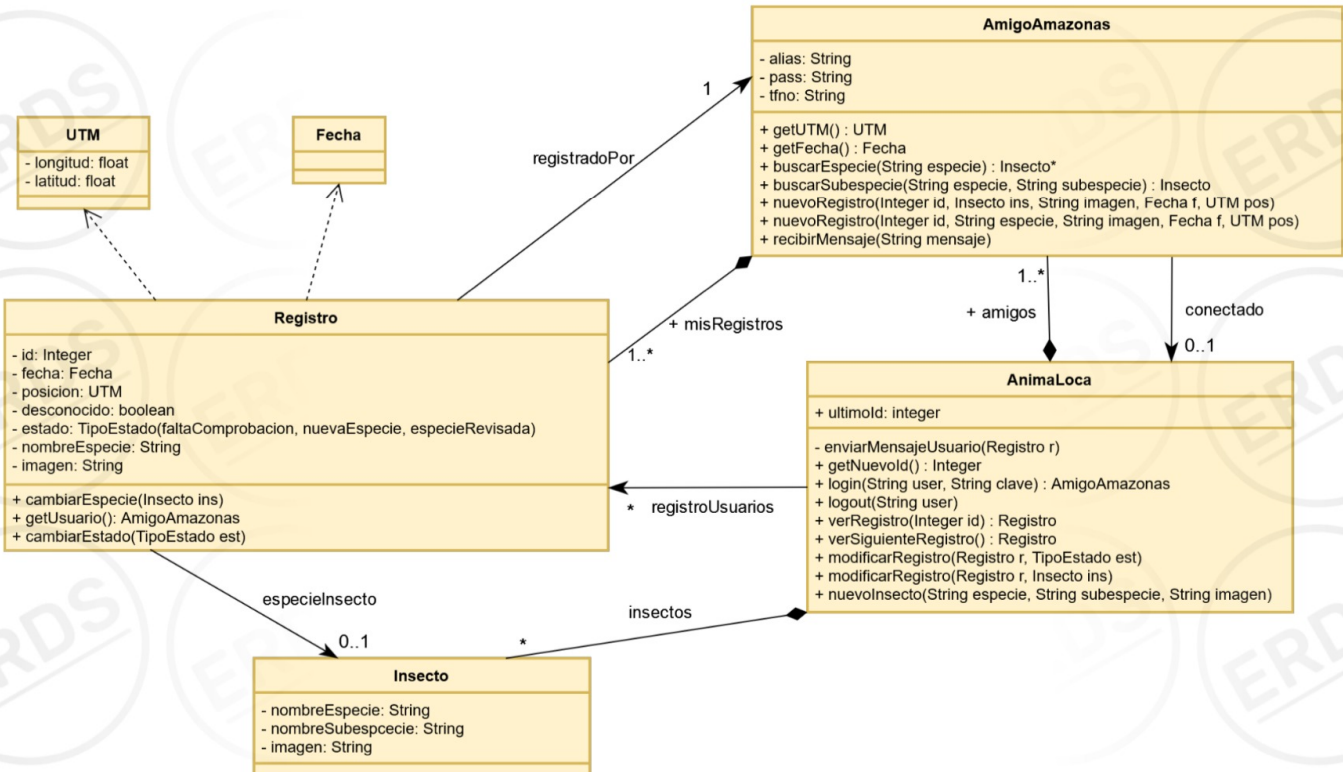
Registro: cada vez que un amigo del sistema ve un insecto que le parece interesante, le hace una fotografía con su móvil para crear un registro de dicho insecto. El registro toma un id automáticamente del sistema. La posición del GPS y la fecha son importantes pero se cogen directamente del móvil del usuario representado por la clase AmigoAmazonas. Un usuario siempre conoce la especie, pero si no ha conseguido conocer la subespecie, entonces no se establece una conexión con el insecto, sino que se pone el atributo *desconocido*=true y se espera a que los expertos lo cataloguen. Siempre que el usuario crea un nuevo informe el *estado*=faltaComprobación. Cuando los expertos lo visualizan finalmente determinarán que la especie está bien catalogada (*EspecieRevisada*) o que por el contrario la especie es nueva lo que se establece como *NuevaEspecie*. En cualquier momento el experto puede observar que el insecto catalogado sea erróneo y lo puede modificar con *cambiarEspecie()*. También se puede obtener el usuario que ha realizado el registro con *getUsuario()*.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

AmigoAmazonas: representa a cada uno de los voluntarios del proyecto conectado con su teléfono móvil. Tiene un *alias* y *clave* para entrar al sistema y un número de *teléfono*. Cuando está conectado se crea un vínculo con la clase principal *AnimaLoca* y cuando se desconecta este vínculo desaparece. Las operaciones de *getUTM()* y *getFecha()* se toman de su móvil y permiten conocer la posición y fecha del registro (no se implementan). Este usuario, cuando localiza un ejemplar y consigue hacer la foto, puede clasificarlo directamente consultando el catálogo de insectos a través de la aplicación mediante las operaciones *buscarEspecie()* (el * simboliza muchos) y *buscarSubespecie()*. Si consigue reconocer la subespecie realizará un registro con el insecto específico tras localizarlo con las operaciones anteriores, sino identificará tan solo la especie (*Registro::nombreEspecie*) y realizará el registro de forma que quede catalogado como desconocido.

AnimaLoca: Es la clase principal de la aplicación cuyo objetivo e **animar** a la **localización** de **animales**, en particular de insectos. Genera un nuevo *Id* para cada registro de forma automática (se comienza por 1). Permite logear a un usuario y desconectarlo del sistema. Puede obtener registros por su identificador con *verRegistro()*, o bien el siguiente registro al que le falte comprobación (*Registro::estado=faltaComprobacion*) con *verSiguienteRegistro()*. También permite modificar el estado de un registro (*modificarRegistro()*) porque se corrobore que es correcto su catalogación (*Registro::estado=EspecieRevisada*) o por el contrario porque sea errónea y se corrija cambiando el insecto asociado. Este insecto podría ser incluso una nueva especie, que se añadiría al sistema con *nuevoInsecto()*. En todos los casos, cuando un registro se modifica de alguna manera se le envía un mensaje al usuario que hizo dicho registro a través de la función *enviarMensaje()* indicándole si se ha identificado una nueva especie, o si su catalogación fue o no correcta.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional